

✓ Deep Learning — Assignment 11

Assignment for week 11 of the 2023 Deep Learning course (NWI-IMC070) of the Radboud University.

Names: Vojko Pjanovic (s1084416); Carmen Gascó Gálvez (s1098566)

Group: 10

Instructions:

- Fill in your names and the name of your group.
- Answer the questions and complete the code where necessary.
- Keep your answers brief, one or two sentences is usually enough.
- Re-run the whole notebook before you submit your work.
- Save the notebook as a PDF and submit that in Brightspace together with the `.ipynb` notebook file.
- The easiest way to make a PDF of your notebook is via File > Print Preview and then use your browser's print option to print to PDF.

Objectives

In this assignment you will

1. Build a diffusion model
2. Extend the model to a class-conditional version

✓ Required software

As before you will need these libraries:

- `torch` and `torchvision` for PyTorch,
- `d2l`, the library that comes with [Dive into deep learning](#) book.

All libraries can be installed with `pip install`.

```
!pip install d2l
```

Requirement already satisfied: d2l in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: jupyter==1.0.0 in /usr/local/lib/python3.10/
Requirement already satisfied: numpy==1.23.5 in /usr/local/lib/python3.10/d
Requirement already satisfied: matplotlib==3.7.2 in /usr/local/lib/python3.
Requirement already satisfied: matplotlib-inline==0.1.6 in /usr/local/lib/p
Requirement already satisfied: requests==2.31.0 in /usr/local/lib/python3.1
Requirement already satisfied: pandas==2.0.3 in /usr/local/lib/python3.10/d
Requirement already satisfied: scipy==1.10.1 in /usr/local/lib/python3.10/d
Requirement already satisfied: notebook in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: qtconsole in /usr/local/lib/python3.10/dist-
Requirement already satisfied: jupyter-console in /usr/local/lib/python3.10
Requirement already satisfied: nbconvert in /usr/local/lib/python3.10/dist-
Requirement already satisfied: ipykernel in /usr/local/lib/python3.10/dist-
Requirement already satisfied: ipywidgets in /usr/local/lib/python3.10/dist
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.1
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.10/di
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/d
Requirement already satisfied: pyparsing<3.1,>=2.3.1 in /usr/local/lib/pyth
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/pytho
Requirement already satisfied: traitlets in /usr/local/lib/python3.10/dist-
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/di
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.10/
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/p
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/di
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: ipython-genutils in /usr/local/lib/python3.1
Requirement already satisfied: ipython>=5.0.0 in /usr/local/lib/python3.10/
Requirement already satisfied: jupyter-client in /usr/local/lib/python3.10/
Requirement already satisfied: tornado>=4.2 in /usr/local/lib/python3.10/di
Requirement already satisfied: widgetsnbextension~=3.6.0 in /usr/local/lib/
Requirement already satisfied: jupyterlab-widgets>=1.0.0 in /usr/local/lib/
Requirement already satisfied: prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0
Requirement already satisfied: pygments in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: lxml in /usr/local/lib/python3.10/dist-packa
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.10/
Requirement already satisfied: bleach in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: defusedxml in /usr/local/lib/python3.10/dist
Requirement already satisfied: entrypoints>=0.2.2 in /usr/local/lib/python3
Requirement already satisfied: jinja2>=3.0 in /usr/local/lib/python3.10/dis
Requirement already satisfied: jupyter-core>=4.7 in /usr/local/lib/python3.
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10
Requirement already satisfied: mistune<2,>=0.8.1 in /usr/local/lib/python3.
Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.10
Requirement already satisfied: nbformat>=5.1 in /usr/local/lib/python3.10/d
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/pytho
Requirement already satisfied: tinycss2 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: pyzmq<25,>=17 in /usr/local/lib/python3.10/d
Requirement already satisfied: argon2-cffi in /usr/local/lib/python3.10/dis
Requirement already satisfied: nest-asyncio>=1.5 in /usr/local/lib/python3.
Requirement already satisfied: Send2Trash>=1.8.0 in /usr/local/lib/python3.
Requirement already satisfied: terminado>=0.8.3 in /usr/local/lib/python3.1

Requirement already satisfied: prometheus-client in /usr/local/lib/python3.10.10/dist-packages
Requirement already satisfied: nbclassic>=0.4.7 in /usr/local/lib/python3.10.10/dist-packages
Requirement already satisfied: starlette<0.40.0 in /usr/local/lib/python3.10.10/dist-packages

```
%config InlineBackend.figure_formats = ['png']  
%matplotlib inline
```

```
from d2l import torch as d2l  
import itertools  
import numpy as np  
import matplotlib.pyplot as plt  
import torch  
from torch import nn  
from torch.nn import functional as F  
from torchvision import datasets, transforms
```

✓ 11.1 MNIST dataset

In this assignment we will once again use the MNIST digit dataset. This dataset consists of 28×28 binary images and has 60000 training examples divided over 10 classes. We split this into 55000 images for training and 5000 images for validation.

As preprocessing, we pad the images to 32x32 pixels and scale the intensities to [-1, +1].

(a) Run the code below to load the MNIST dataset.

```
transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Pad(2),          # pad to 32x32 pixels  
    transforms.Normalize(0.5, 0.5), # normalize to [-1, +1]  
)  
train_val_data = datasets.MNIST('data', train=True, download=True, transform=tr  
# Split into train and validation set  
train_data, validation_data = torch.utils.data.random_split(train_val_data, [55  
# Create data loaders  
train_iter      = torch.utils.data.DataLoader(train_data, batch_size=128, shuf  
validation_iter = torch.utils.data.DataLoader(validation_data, batch_size=1000)  
num_classes = 10
```

✓ 11.2 Training images (4 points)

We will implement a model from the paper [Denoising Diffusion Probabilistic Models](#) by Ho et al., 2020.

We reuse some parameter settings from the paper:

```
diffusion_steps = 1000
beta = torch.linspace(1e-4, 0.02, diffusion_steps)
alpha = 1.0 - beta
alpha_bar = torch.cumprod(alpha, dim=0)
```

Using these settings, we can generate a sequence of noisy images with

$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1} \quad \text{where } \epsilon_{t-1}, \epsilon_{t-2}, \dots \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

There is a closed-form solution to compute x_t directly from x_0 (see the paper or [this blog](#)):

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \quad \text{where } \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

(a) Implement this closed-form solution in the code below to generate random images. (1 point)

Expected output: you should see some images with recognizable shapes and some images with noise.

```
# sample some original images
x_0, y = next(iter(train_iter))

plt.figure(figsize=(19, 21))
for i, t in enumerate(range(0, diffusion_steps, 100)):
    # generate random noise
    noise = torch.randn_like(x_0)

    # add noise to the original image to compute x_t
    x_t = torch.sqrt(alpha_bar[t]) * x_0 + torch.sqrt(1 - alpha_bar[t]) * noise

    plt.subplot(3, diffusion_steps // 100, i + 1)
    plt.imshow(x_t[0, 0], cmap='gray')
    plt.axis('off')
    plt.title(f't={t}')
plt.tight_layout()
plt.show()
```

(b) Describe how we should interpret these images, and how they can be used to train the diffusion model. (2 points)

As t increases, noise is added at each step, making the last one just random noise. The first one is the original image.

The goal of the diffusion model is to proceed in a reverse way, starting from a noisy image, and removing the noise at each step, until we get the original image.

During training, we will need a minibatch with multiple images and multiple time steps.

(c) Complete the function below to add noise to a minibatch of images. (1 point)

```
def generate_noisy_samples(x_0, beta):  
    """  
    Create noisy samples for the minibatch x_0.  
    Return the noisy image, the noise, and the time for each sample.  
    """  
    alpha = 1.0 - beta  
    alpha_bar = torch.cumprod(alpha, dim=0)  
  
    # sample a random time t for each sample in the minibatch  
    t = torch.randint(beta.shape[0], size=(x_0.shape[0],), device=x_0.device)  
    # generate noise  
    noise = torch.randn_like(x_0)  
    # add the noise to each sample  
    x_t = torch.sqrt(alpha_bar[t, None, None, None]) * x_0 + torch.sqrt(1 - alpha_bar[t, None, None, None]) * noise  
    return x_t, noise, t
```

(d) Try out your new function by generating a few noisy samples.

Expected output: you should see the samples in the minibatch with different levels of noise, depending on the time t for each sample.

```

x_0, y = next(iter(train_iter))

x_t, noise, sampled_t = generate_noisy_samples(x_0, beta)
assert x_t.shape == x_0.shape
assert noise.shape == x_0.shape
assert sampled_t.shape == (x_0.shape[0],)

plt.figure(figsize=(12, 6))
for i in range(32):
    plt.subplot(4, 8, i + 1)
    plt.imshow(x_t[i, 0], cmap='gray')
    plt.axis('off')
    plt.title(f't={sampled_t[i]}')
plt.tight_layout()

```

✓ 11.3 Helper functions

We will use some predefined components to construct our model, based on [an existing implementation](#) on GitHub.

```

class SelfAttention(nn.Module):
    def __init__(self, h_size):
        super(SelfAttention, self).__init__()
        self.h_size = h_size
        self.mha = nn.MultiheadAttention(h_size, 4, batch_first=True)
        self.ln = nn.LayerNorm([h_size])
        self.ff_self = nn.Sequential(
            nn.LayerNorm([h_size]),
            nn.Linear(h_size, h_size),
            nn.GELU(),
            nn.Linear(h_size, h_size),
        )

    def forward(self, x):
        x_ln = self.ln(x)
        attention_value, _ = self.mha(x_ln, x_ln, x_ln)
        attention_value = attention_value + x
        attention_value = self.ff_self(attention_value) + attention_value
        return attention_value

class SAWrapper(nn.Module):
    def __init__(self, h_size, num_s):
        super(SAWrapper, self).__init__()
        self.sa = nn.Sequential(*[SelfAttention(h_size) for _ in range(1)])
        self.num_s = num_s
        self.h_size = h_size

```

```

def forward(self, x):
    x = x.view(-1, self.h_size, self.num_s * self.num_s).swapaxes(1, 2)
    x = self.sa(x)
    x = x.swapaxes(2, 1).view(-1, self.h_size, self.num_s, self.num_s)
    return x

```

U-Net code adapted from: <https://github.com/milesial/Pytorch-UNet>

```

class DoubleConv(nn.Module):
    def __init__(self, in_channels, out_channels, mid_channels=None, residual=False):
        super().__init__()
        self.residual = residual
        if not mid_channels:
            mid_channels = out_channels
        self.double_conv = nn.Sequential(
            nn.Conv2d(in_channels, mid_channels, kernel_size=3, padding=1, bias=True),
            nn.GroupNorm(1, mid_channels),
            nn.GELU(),
            nn.Conv2d(mid_channels, out_channels, kernel_size=3, padding=1, bias=True),
            nn.GroupNorm(1, out_channels),
        )

```

```

    def forward(self, x):
        if self.residual:
            return F.gelu(x + self.double_conv(x))
        else:
            return self.double_conv(x)

```

```

class Down(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.maxpool_conv = nn.Sequential(
            nn.MaxPool2d(2),
            DoubleConv(in_channels, in_channels, residual=True),
            DoubleConv(in_channels, out_channels),
        )

```

```

    def forward(self, x):
        return self.maxpool_conv(x)

```

```

class Up(nn.Module):
    def __init__(self, in_channels, out_channels, bilinear=True):
        super().__init__()

        # if bilinear, use the normal convolutions to reduce the number of channels
        if bilinear:
            self.up = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=True)
            self.conv = DoubleConv(in_channels, in_channels, residual=True)
            self.conv2 = DoubleConv(in_channels, out_channels, in_channels // 2)
        else:

```

```

        self.up = nn.ConvTranspose2d(
            in_channels, in_channels // 2, kernel_size=2, stride=2
        )
        self.conv = DoubleConv(in_channels, out_channels)

def forward(self, x1, x2):
    x1 = self.up(x1)
    # input is CHW
    diffY = x2.size()[2] - x1.size()[2]
    diffX = x2.size()[3] - x1.size()[3]

    x1 = F.pad(x1, [diffX // 2, diffX - diffX // 2, diffY // 2, diffY - diffY // 2])
    x = torch.cat([x2, x1], dim=1)
    x = self.conv(x)
    x = self.conv2(x)
    return x

class OutConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super(OutConv, self).__init__()
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=1)

    def forward(self, x):
        return self.conv(x)

```

✓ 11.4 Diffusion model (5 points)

Similar to Ho et al. and to several online implementations, we will use a U-Net with self-attention and positional embedding as our diffusion model.

(a) Familiarize yourself with the architecture of this U-Net.

```

#device = d2l.try_gpu()
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```



```

class UNet(nn.Module):
    def __init__(self, c_in=1, c_out=1, device= device): #
        super().__init__()
        self.device = device

        bilinear = True
        self.inc = DoubleConv(c_in, 64)
        self.down1 = Down(64, 128)
        self.down2 = Down(128, 256)
        factor = 2 if bilinear else 1
        self.down3 = Down(256, 512 // factor)
        self.up1 = Up(512, 256 // factor, bilinear)
        self.up2 = Up(256, 128 // factor, bilinear)
        self.up3 = Up(128, 64, bilinear)
        self.outc = OutConv(64, c_out)
        self.sa1 = SAWrapper(256, 8)
        self.sa2 = SAWrapper(256, 4)
        self.sa3 = SAWrapper(128, 8)

    def pos_encoding(self, t, channels, embed_size):
        inv_freq = 1.0 / (
            10000
            ** (torch.arange(0, channels, 2, device=self.device).float() / char
            )
        pos_enc_a = torch.sin(t[:, None].repeat(1, channels // 2) * inv_freq)
        pos_enc_b = torch.cos(t[:, None].repeat(1, channels // 2) * inv_freq)
        pos_enc = torch.cat([pos_enc_a, pos_enc_b], dim=-1)
        return pos_enc.view(-1, channels, 1, 1).repeat(1, 1, embed_size, embed_

    def forward(self, x, t):
        """
        Model is U-Net with added positional encodings and self-attention layer
        """
        x1 = self.inc(x)
        x2 = self.down1(x1) + self.pos_encoding(t, 128, 16)
        x3 = self.down2(x2) + self.pos_encoding(t, 256, 8)
        x3 = self.sa1(x3)
        x4 = self.down3(x3) + self.pos_encoding(t, 256, 4)
        x4 = self.sa2(x4)
        x = self.up1(x4, x3) + self.pos_encoding(t, 128, 8)
        x = self.sa3(x)
        x = self.up2(x, x2) + self.pos_encoding(t, 64, 16)
        x = self.up3(x, x1) + self.pos_encoding(t, 64, 32)
        output = self.outc(x)
        return output

```

(b) What does the positional encoding encode? Why would this be useful? (2 points)

Your answer here on what it encodes.

It encodes the time step, so that the model can learn to remove the noise at each step.

Your answer here on why it is useful.

It is useful because it allows the model to learn to remove the noise at each step. Knowing that the images will be noisier at each step, the model can learn to remove the noise.

(c) Describe how this model will be used. What do the inputs and outputs represent? (3 points)

Used for: Determining the noise added at the previous step, so that it can be removed. Hence, after t iterations, the original image would be obtained.

Inputs: The image and the corresponding time step t .

Outputs: The added noise at each step.

✓ 11.5 Training the model (7 points)

We will train our diffusion model using Algorithm 1 from the paper [Denoising Diffusion Probabilistic Models](#) by Ho et al., 2020.

(a) The algorithm uses x_0 . How do you obtain x_0 during training? (1 point)

x_0 is obtained as a random image from the training set.

(b) Which two values are compared in the loss on line 5 of the algorithm? (1 point)

The loss is computed comparing the actual random noise value with the estimated by the network.

(c) Implement this procedure in the training loop below. (3 points)

```

def train(model, beta, num_epochs=10, lr=1e-3, device= device):
    """Train a diffusion model"""
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    animator = d2l.Animator(xlabel='epoch', xlim=[1, num_epochs], figsize=(10,
                                legend=['train loss', 'validation loss']))
    for epoch in range(num_epochs):
        metric = d2l.Accumulator(2)
        model.train()
        print('epoch', epoch)
        for x, y in train_iter:
            x = x.to(device)
            optimizer.zero_grad()
            # compute the loss for minibatch x
            # hint: we already have a function to generate noisy images
            x_t, noise, sampled_t = generate_noisy_samples(x, beta.to(device))
            estimated_noise = model(x_t, sampled_t.to(torch.float))
            loss = F.mse_loss(estimated_noise, noise)

            # Optimize
            loss.backward()
            optimizer.step()

            # Track our progress
            metric.add(loss.detach() * x.shape[0], x.shape[0])
        train_loss = metric[0] / metric[1]

        # Compute validation loss
        validation_loss = test(model, validation_iter, beta)

        # Plot
        animator.add(epoch + 1, (train_loss, validation_loss))
    print(f'training loss {train_loss:.3g}, validation loss {validation_loss:.3g}')

def test(model, test_iter, beta, device=device):
    """Test a diffusion model by computing the loss."""
    metric = d2l.Accumulator(2)
    model.eval()
    for x, y in test_iter:
        x = x.to(device)
        with torch.no_grad():
            # compute the loss for minibatch x
            x_t, noise, sampled_t = generate_noisy_samples(x, beta.to(device))
            estimated_noise = model(x_t, sampled_t.to(torch.float))
            loss = F.mse_loss(estimated_noise, noise)
            metric.add(loss.detach() * x.shape[0], x.shape[0])
    return metric[0] / metric[1]

```

(d) How does the training time depend on the number of diffusion steps T ? (1 point)

Training doesn't depend on the number of T , because the inputs are the two different images, not the entire sequence of T s.

(e) Train the model.

Expected output: in our implementation, the training loss started at around 0.1 and went down quickly to 0.03 and lower.

```
device = d2l.try_gpu()
#device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = UNet().to(device)
train(model, beta, num_epochs=10, lr=1e-4)
```

```
training loss 0.0223, validation loss 0.0219
```

(f) Has the training converged? Do you think we should train longer? (1 point)

The training does not appear to have converged yet because the losses still show a decreasing tendency. Therefore, we should train longer. However, the losses are already very low, so it might be that the model is already good enough and close to convergence.

✓ 11.6 Sampling from the model (9 points)

Once the model is trained, we can sample from it using Algorithm 2 from paper [Denoising Diffusion Probabilistic Models](#):

Algorithm 2:

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: for $t = T, \dots, 1$ do
- 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
- 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\bar{a}_t}} \left(\mathbf{x}_t - \frac{1-a_t}{\sqrt{1-\bar{a}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: end for
- 6: return $\mathbf{x}_0$

(a) In step 3 of algorithm 2, $\mathbf{z}$ is set to $\mathbf{0}$ some times. What is the effect of this? (1 point)

In this case, Z is the noise added at each step. If it is set to 0, it means that no noise is added, so the output is the original image.

(b) In step 4 of algorithm 2, $\mathbf{x}_{t-1}$ is computed based on three ingredients: $\mathbf{x}_t$, $\epsilon_\theta(\mathbf{x}_t, t)$, and $\mathbf{z}$. What do these represent? (2 points)

- $\mathbf{x}_t$: noisy image at step t .
- $\epsilon_\theta(\mathbf{x}_t, t)$: noise estimated at step t , according to the model θ .
- $\mathbf{z}$: noise added.

(c) How does the sampling time depend on the number of diffusion steps T ? (1 point)

The iterations start with $t = T$, and go down to $t = 0$, so the sampling time depends linearly with T .

(d) Complete the code below to sample a minibatch from the model. (2 points)

- Use the equations in Algorithm 2.
- Use $\sigma_t = \sqrt{\beta_t}$, as suggested in the paper.
- Keep in mind that Algorithm 2 uses $t = 1$ as the first time step, whereas we use $t = 0$.

Expected output: after training, your model should generate fairly realistic, clean images when given random inputs.

```

def sample_from_model(x, model, beta):
    # keep track of x at different time steps
    x_hist = []
    with torch.no_grad():
        # loop over all time steps in reverse order
        for i in reversed(range(0, beta.shape[0])):
            # copy the time step for each sample in the minibatch
            t = (torch.ones(x.shape[0]) * i).long().to(device)
            # compute the next value of x
            z = torch.randn_like(x) if i > 0 else torch.zeros_like(x)
            sigma = torch.sqrt(beta[i])
            x = (1 / torch.sqrt(alpha[i])) * (x - ((1 - alpha[i]) / torch.sqrt(
                if i % 100 == 0:
                    x_hist.append(x.detach().cpu().numpy())
    return x, x_hist

def plot_x_hist(x_hist):
    # plot the generated images
    plt.figure(figsize=(10, 10))
    for i in range(len(x_hist)):
        for j in range(10):
            plt.subplot(10, 10, j * 10 + i + 1)
            plt.imshow(x_hist[i][j, 0])
            plt.axis('off')

# initialize x with the right values
# shape: [10, 1, 32, 32]
x = torch.randn_like(x_0[:10]).to(device)

x, x_hist = sample_from_model(x, model, beta)
plot_x_hist(x_hist)

```

(e) Explain the X and Y axes of this figure. (1 point)

The X axis represents the intermediate steps t , and the Y axis the examples of different samples (labels).

(f) In a variational autoencoder or a GAN, the output is determined by the latent representation. How does that work for this diffusion model? (1 point)

In this case there is not a clear representation of the latent space, the procedure here is to denoise the image with each step.

(g) Look at the generated intermediate samples over time in question (d). Do we need all of the steps? Why/why not? (1 point)

Yes, we need a reasonable number of steps to get a good result. As the model denoises the image at each step, the more steps we have, the better the result will be.

✓ 11.7 Experiments (5 points)

✓ Fixed initialization

How does the end result depend on the initialization? We will generate multiple images from the same initial noise to find out.

(a) Complete and run the code below.

Expected output: the model should still produce recognizable shapes.

```
# initialize x=x_T with the right values for one image
# (see earlier question)
# shape: [1, 1, 32, 32]
x = torch.randn_like(x_0[:1]).to(device)
# repeat this to generate 10 images from the same initialization
x = x.repeat(10, 1, 1, 1)

x, x_hist = sample_from_model(x, model, beta)
plot_x_hist(x_hist)
```

(b) Does the model always produce the same output from the same initial input? Why, or why not? (2 points)

No, because the noise is added randomly at each step, so the result will be different.

✓ No randomness between time steps

To check the influence of noise during sampling, we can remove the term $\sigma_t \mathbf{z}$ from Algorithm 2.

(c) Create a new function `deterministic_sample_from_model`, based on `sample_from_model`, that does not include this term. (2 points)

```
def deterministic_sample_from_model(x, model, beta):
    # copy sample_from_model and remove the random term sigma * z
    # keep track of x at different time steps
    x_hist = []
    with torch.no_grad():
        # loop over all time steps in reverse order
        for i in reversed(range(0, beta.shape[0])):
            # copy the time step for each sample in the minibatch
            t = (torch.ones(x.shape[0]) * i).long().to(device)
            # compute the next value of x
            z = torch.randn_like(x) if i > 0 else torch.zeros_like(x)
            sigma = torch.sqrt(beta[i])
            x = (1 / torch.sqrt(alpha[i])) * (x - ((1 - alpha[i]) / torch.sqrt(
                if i % 100 == 0:
                    x_hist.append(x.detach().cpu().numpy())
    return x, x_hist
```

(d) Generate some samples using the new function.

Expected output: you should get a different result than before.


```
# initialize x with the right values
# (see earlier questions)
# shape: [10, 1, 32, 32]
x = torch.randn_like(x_0[:10]).to(device)

x, x_hist = deterministic_sample_from_model(x, model, beta)
plot_x_hist(x_hist)
```

(e) What can you conclude from these results? Is the random noise during sampling important? (1 point)

As the model takes as a given that there is noise at the different steps, if we remove them the results become meaningless.

✓ 11.8 Making the model conditional (6 points)

Similar to the conditional VAE in the previous assignment, we can make the diffusion model conditional by including class labels. This allows us to generate samples from a specific digit.

We will include the class information alongside the existing positional encoding, using a [torch.nn.Embedding](#) layer to map the 10 digits to a higher-dimensional space.

✓ Conditional model

(a) Study the implementation of UNetConditional to see how this works.

```
class UNetConditional(nn.Module):
    def __init__(self, c_in=1, c_out=1, n_classes=10, device=device):
        super().__init__()
        self.device = device

        bilinear = True
        self.inc = DoubleConv(c_in, 64)
        self.down1 = Down(64, 128)
        self.down2 = Down(128, 256)
        factor = 2 if bilinear else 1
        self.down3 = Down(256, 512 // factor)
        self.up1 = Up(512, 256 // factor, bilinear)
        self.up2 = Up(256, 128 // factor, bilinear)
        self.up3 = Up(128, 64, bilinear)
        self.outc = OutConv(64, c_out)
        self.sa1 = SAWrapper(256, 8)
        self.sa2 = SAWrapper(256, 4)
        self.sa3 = SAWrapper(128, 8)

        self.label_embedding = nn.Embedding(n_classes, 256)

    def pos_encoding(self, t, channels, embed_size):
        inv_freq = 1.0 / (
            10000
```

```

        ** (torch.arange(0, channels, 2, device=self.device).float() / char
    )
    pos_enc_a = torch.sin(t[:, None].repeat(1, channels // 2) * inv_freq)
    pos_enc_b = torch.cos(t[:, None].repeat(1, channels // 2) * inv_freq)
    pos_enc = torch.cat([pos_enc_a, pos_enc_b], dim=-1)
    return pos_enc.view(-1, channels, 1, 1).repeat(1, 1, embed_size, embed_

def label_encoding(self, label, channels, embed_size):
    return self.label_embedding(label)[: , :channels, None, None].repeat(1,

def forward(self, x, t, label):
    """
    Model is U-Net with added positional encodings and self-attention layer
    """
    x1 = self.inc(x)
    x2 = self.down1(x1) + self.pos_encoding(t, 128, 16) + self.label_encodi
    x3 = self.down2(x2) + self.pos_encoding(t, 256, 8) + self.label_encodir
    x3 = self.sa1(x3)
    x4 = self.down3(x3) + self.pos_encoding(t, 256, 4) + self.label_encodir
    x4 = self.sa2(x4)
    x = self.up1(x4, x3) + self.pos_encoding(t, 128, 8) + self.label_encodi
    x = self.sa3(x)
    x = self.up2(x, x2) + self.pos_encoding(t, 64, 16) + self.label_encodir
    x = self.up3(x, x1) + self.pos_encoding(t, 64, 32) + self.label_encodir
    output = self.outc(x)
    return output

```

(b) As in [the paper by Ho et al.](#), the position and label encoding are added in every layer of the model, instead of as an input to the first layer only. Why do you think the authors made this choice? (1 point)

If the positional and label encodings are added at every layer on t in order, this could be useful to keep track at each step of the noising process.

✓ Conditional training loop

(c) Create a new function `train_conditional` to train this model. (1 point)

```

def train_conditional(model, beta, num_epochs=10, lr=1e-3, device= device):
    # implement the training loop, based on your earlier train function
    """Train a diffusion model"""
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    animator = d2l.Animator(xlabel='epoch', xlim=[1, num_epochs], figsize=(10,
                                                                    legend=['train loss', 'validation loss']))
    for epoch in range(num_epochs):
        metric = d2l.Accumulator(2)
        model.train()
        print('epoch', epoch)
        for x, y in train_iter:
            x = x.to(device)
            y = y.to(device)
            optimizer.zero_grad()
            # compute the loss for minibatch x
            # hint: we already have a function to generate noisy images
            x_t, noise, sampled_t = generate_noisy_samples(x, beta.to(device))
            estimated_noise = model(x_t, sampled_t.to(torch.float), y)
            loss = F.mse_loss(estimated_noise, noise)

            # Optimize
            loss.backward()
            optimizer.step()

            # Track our progress
            metric.add(loss.detach(), x.shape[0])
        train_loss = metric[0] / metric[1]

        # Compute validation loss
        # validation_loss = test(model, validation_iter, beta)

        metric = d2l.Accumulator(2)
        model.eval()
        for x, y in validation_iter:
            x = x.to(device)
            y = y.to(device)
            with torch.no_grad():
                # compute the loss for minibatch x
                x_t, noise, sampled_t = generate_noisy_samples(x, beta.to(device))
                estimated_noise = model(x_t, sampled_t.to(torch.float), y)
                loss = F.mse_loss(estimated_noise, noise)
                metric.add(loss.detach(), x.shape[0])
        test_loss = metric[0] / metric[1]

        # Plot
        animator.add(epoch + 1, (train_loss, test_loss))
    print(f'training loss {train_loss:.3g}, validation loss {test_loss:.3g}')

```

(d) Train the conditional model.

Expected output: in our implementation, the training loss started at around 0.1 and went down quickly to 0.03 and lower.

```
model_conditional = UNetConditional().to(device)
train_conditional(model_conditional, beta, num_epochs=10, lr=1e-4)
```

```
training loss 0.000171, validation loss 2.26e-05
```

✓ Conditional sampling

(e) Modify the sampling function to include a label. (1 point)

```

def sample_from_model_conditional(x, model, beta, label, device= device):
    # copy and modify sample_from_model to include the label
    # keep track of x at different time steps
    x_hist = []
    with torch.no_grad():
        aux = (torch.ones(x.shape[0]) * label).long().to(device)
        # loop over all time steps in reverse order
        for i in reversed(range(0, beta.shape[0])):
            # copy the time step for each sample in the minibatch
            t = (torch.ones(x.shape[0]) * i).long().to(device)
            # compute the next value of x
            z = torch.randn_like(x) if i > 0 else torch.zeros_like(x)
            sigma = torch.sqrt(beta[i])
            x = (1 / torch.sqrt(alpha[i])) * (x - ((1 - alpha[i]) / torch.sqrt(
                if i % 100 == 0:
                    x_hist.append(x.detach().cpu().numpy())
    return x, x_hist

```

(f) Sample a few digits with label 3 . (1 point)

Expected output: you should see recognizable images with the number you requested.

```
# sample some digits
x = torch.randn_like(x_0[:10]).to(device)
x, x_hist = sample_from_model_conditional(x, model_conditional, beta, label=3)
plot_x_hist(x_hist)
```

(g) Complete the code to sample and plot 10 samples for every digit. (1 point)

```
x_per_class = []  
for label in range(10):  
    # sample 10 digits with this label  
    x = torch.randn_like(x_0[:10]).to(device)  
    x, x_hist = sample_from_model_conditional(x, model_conditional, beta, label)  
    x_per_class.append(x.detach().cpu().numpy())
```



```
plt.figure(figsize=(10, 10))
for i in range(10):
    for j in range(10):
        plt.subplot(10, 10, j * 10 + i + 1)
        plt.imshow(x_per_class[j][i, 0])
        plt.axis('off')
```

(h) Compare the output of the conditional model with that of the unconditional model. Which one is better? (1 point)

The conditional model reaches lower losses than the unconditional model, and it also seems to obtain images with more definition. Therefore, we can infer that its performance is slightly better.

✓ 11.9 Discussion (6 points)

(a) Compare the sources of randomness in our diffusion model with that in the variational autoencoder and the GAN in earlier assignments. What are the main differences? (1 point)

In the diffusion model, the randomness is added at each step, and it is not clear what the latent space is. In the VAE, the randomness is added at the latent space, and in the GAN, the randomness is added at the input.

(b) Would you be able to train a good digit classification model on the initial input to the sampling function? Why, or why not? (1 point)

Hint: for variational autoencoders and GANs, there is a clear link between the input (a latent feature vector) and the output of the decoder. How does this work for our diffusion model?

The result of this model has a highly dependency on the noise. Therefore, training on the initial input to the sampling function would lead to problems of denoising, as we could not be able to keep track of the removed noise and the current T step.

(c) When loading the data, we normalized the image intensities to $[-1, +1]$, instead of $[0, 1]$ or $[0, 255]$. Why is this a good input range for this diffusion model? (1 point)

The image data are integers of $[0, 255]$, whereas the noise is taken from a random normal distribution $(0,1)$. To be able to add the noise to the image, we need to have the same range of values, and that is why these values are normalized to $[-1, +1]$.

(d) In this assignment, we use a β schedule that has a small $\beta = 1e - 4$ at the initial time steps ($t = 0$), and a larger $\beta = 0.02$ at the end ($t = T$). Why is it useful to choose an increasing β ? (1 point)

Beta is increasing with t and therefore with the presence of noise. This parameter is used to control the amount of noise added at each step. As the noise increases, the model needs to learn to remove more noise, so it is useful to increase beta.

(e) What would happen if we made β very small? (1 point)

The model would need a higher number of T steps to reach the same output.

(f) What would happen if we made β very large? (1 point)

The result would be achieved earlier, but it would be worse than the ones obtained with a smaller beta.

✓ The end

Well done! Please double check the instructions at the top before you submit your results.

This assignment has 42 points. Version 55ce6f4 / 2023-11-30

